Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-213БВ-25

Студент: Дарвин Ю.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 11.10.26

Москва, 2026

Постановка задачи

Вариант 6.

Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на чтение. Стандартный поток ввода дочернего процесса переопределяется открытым файлом. Дочерний процесс читает команды из стандартного потока ввода. Стандартный поток вывода дочернего процесса перенаправляется в pipe1. Родительский процесс читает из pipe1 и прочитанное выводит в свой стандартный поток вывода. Родительский и дочерний процесс должны быть представлены разными программами.

В файле записаны команды вида: «число число число<endline>». Дочерний процесс считает их сумму и выводит результат в стандартный поток вывода. Числа имеют тип int. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); – создает дочерний процесс.
- int pipe(int *fd); - создает однонаправленный канал для обмена данными между процессами
- int dup2(int oldfd, int newfd); - создает копию файлового дескриптора
- int execl(const char *path, const char *arg0, const char *arg1, …, (char *)NULL); - заменяет текущий образ процесса новым исполняемым файлом
- read
- write

В рамках лабораторной работы были разработаны две взаимосвязанные программы на языке C (родительская и дочерняя) для демонстрации межпроцессорного взаимодействия (IPC).

Работа программы начинается с того, что родительский процесс запрашивает у пользователя имя файла, открывает его в режиме чтения с помощью системного вызова `open` и создает неименованный канал связи (`pipe`) для организации обмена данными. Затем родительский процесс вызывает `fork`, создавая свою точную копию, после чего выполнение кода разделяется на два параллельных потока. В дочернем процессе закрывается ненужный конец канала для чтения, а с помощью системного вызова `dup2` стандартный ввод (`STDIN_FILENO`) перенаправляется на дескриптор открытого файла, и стандартный вывод (`STDOUT_FILENO`) перенаправляется на дескриптор записи в канал. Сразу после этого дочерний процесс заменяет свой текущий образ на исполняемый файл программы `child` с помощью вызова `execl`. Запущенная программа `child` начинает порционно считывать данные из файла через стандартный ввод, корректно обрабатывая ситуации, когда число может быть разорвано между двумя вызовами `read` (благодаря механизму сохранения и дочитывания «хвоста» буфера), проверяет возможные арифметические переполнения типа `int` и записывает итоговую сумму через стандартный вывод прямо в канал. Параллельно с этим родительский процесс закрывает свои ненужные дескрипторы (конец канала для записи и исходный файл), переходит в режим ожидания данных, читает результат из канала с помощью `read` и сразу выводит его на экран терминала через `write`. Завершает работу родительский процесс вызовом `wait`, который блокирует его выполнение до момента корректного завершения дочернего

процесса, тем самым предотвращая появление процессов-зомби и обеспечивая своевременное освобождение системных ресурсов операционной системой.

Код программы

parent.c

```
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    char filename[256];
    int pipefd[2];
    pid_t pid;

    char input_msg[] = "Введите имя файла:\n";
    write(STDOUT_FILENO, input_msg, sizeof(input_msg) - 1);

    int pos = 0;
    char ch;
    while (read(STDIN_FILENO, &ch, 1) > 0 && ch != '\n') {
        filename[pos++] = ch;
    }
    filename[pos] = '\0';

    int file_fd = open(filename, O_RDONLY);
    if (file_fd < 0) {
        write(STDOUT_FILENO, "Ошибка открытия файла\n", 23);
        return 1;
    }

    if (pipe(pipefd) < 0) {
        write(STDOUT_FILENO, "Ошибка создания pipe\n", 22);
        close(file_fd);
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        write(STDOUT_FILENO, "Ошибка fork\n", 13);
        close(file_fd);
        close(pipefd[0]);
        close(pipefd[1]);
        return 1;
    }
```

```
    if (pid == 0) {
        close(pipefd[0]);

        dup2(file_fd, STDIN_FILENO);
        close(file_fd);

        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execl("./child", "child", NULL);

        write(STDOUT_FILENO, "Ошибка exec\n", 13);
        return 1;
    } else {
        close(pipefd[1]);
        close(file_fd);

        char buffer[1024];
        int bytes_read;

        while ((bytes_read = read(pipefd[0], buffer, sizeof(buffer))) > 0) {
            write(STDOUT_FILENO, buffer, bytes_read);
        }

        close(pipefd[0]);

        int status;
        wait(&status);
    }

    return 0;
}
```

child.c:

```
#include <unistd.h>
#include <stdlib.h>
#include <ctype.h>
#include <stdbool.h>
#include <limits.h>

#define BUFF_SIZE 256
#define INT_SIZE 16

void print_err(void)
{
    char err_message[] = "Ошибка, сумма чисел выходит за границу числовых значений INT\n";

    write(STDOUT_FILENO, err_message, sizeof(err_message));
}

int main(void)
{
    int total = 0;
    int current = 0;

    int bytes_read;

    bool negative = false;
    bool has_digits = false;

    int pos = 0;
    char buffer[BUFF_SIZE];

    while((bytes_read = read(STDIN_FILENO, buffer + pos, BUFF_SIZE - pos - 1)) > 0) {
        bytes_read += pos;

        for(int i = 0; i < bytes_read; i++) {
            if(isspace(buffer[i])) {
                if(has_digits) {
                    if(negative) current = -current;

                    if(current > 0 && total > INT_MAX - current) {
                        print_err();
                        return 1;
                    } else if(current < 0 && total < INT_MIN - current) {
                        print_err();
                        return 1;
                    }

                    total += current;
                    current = 0;
```

```
                negative = false;
                has_digits = false;
            }

            continue;
        }

        if(buffer[i] == '-') {
            negative = true;
            continue;
        }

        if(buffer[i] == '+') continue;

        if(isdigit(buffer[i])) {
            current *= 10;
            current += buffer[i] - '0';

            has_digits = true;
            continue;
        }
    }

    if(has_digits) {
        int tail_start = bytes_read - 1;

        while(tail_start > 0 && isdigit(buffer[tail_start]))
            tail_start--;

        if(tail_start > 0 && buffer[tail_start] == '-')
            tail_start--;

        pos = bytes_read - tail_start;
        for(int j = 0; j < pos; j++)
            buffer[j] = buffer[tail_start + j];

        current = 0;
        negative = false;
        has_digits = false;
    } else
        pos = 0;
}

if(has_digits) {
    if(negative) current = -current;

    if(current > 0 && total > INT_MAX - current) {
        print_err();
```

```
            return 1;
        } else if(current < 0 && total < INT_MIN - current) {
            print_err();
            return 1;
        }

        total += current;
    }

    int count_nums = 0;

    char tmp[INT_SIZE];
    char *ptr_tmp = tmp;

    negative = (total < 0) ? true : false;
    total = (total < 0) ? -total : total;

    if(total == 0) {
        *ptr_tmp++ = '0';
        count_nums = 1;
    } else {
        while(total > 0) {
            *ptr_tmp++ = '0' + total % 10;
            total /= 10;

            count_nums++;
        }
    }

    ptr_tmp--;

    char result[INT_SIZE];
    char *ptr_result = result;

    if(negative)
        *ptr_result++ = '-';

    for(int i = 0; i < count_nums; i++)
        *ptr_result++ = *ptr_tmp--;

    *ptr_result = '\n';

    write(STDOUT_FILENO, result, count_nums + 1);
    return 0;
}
```

Протокол работы программы

```
user@demo:~/Загрузки/Операционные системы/lab_1$ cat test1.txt
1 2 3 4 5
user@demo:~/Загрузки/Операционные системы/lab_1$ ./parent
Введите имя файла:
test1.txt
15

user@demo:~/Загрузки/Операционные системы/lab_1$ cat test2.txt
-10 -20 -30 -5 15
user@demo:~/Загрузки/Операционные системы/lab_1$ ./parent
Введите имя файла:
test2.txt
-50

user@demo:~/Загрузки/Операционные системы/lab_1$ cat test3.txt
100
200
300
400
500
user@demo:~/Загрузки/Операционные системы/lab_1$ ./parent
Введите имя файла:
test3.txt
1500

user@demo:~/Загрузки/Операционные системы/lab_1$ cat test4.txt
0 0 0 0 0
user@demo:~/Загрузки/Операционные системы/lab_1$ ./parent
Введите имя файла:
test4.txt
0

user@demo:~/Загрузки/Операционные системы/lab_1$ cat test5.txt
-1000000 2000000 -5000000 1500000
user@demo:~/Загрузки/Операционные системы/lab_1$ ./parent
Введите имя файла:
test5.txt
-2500000
```

Вывод

В ходе выполнения лабораторной работы были изучены механизмы межпроцессорного взаимодействия в ОС Linux, а также реализован обмен данными между родительским и дочерним процессами через неименованный канал с использованием низкоуровневых системных вызовов. Основной сложностью при разработке стала корректная обработка потокового чтения, так как вызов `read` может разорвать число на границе буфера, что потребовало написания дополнительного механизма для сохранения и дочитывания «хвоста» данных. Данная работа наглядно продемонстрировала разницу между буферизированным вводом-выводом стандартной библиотеки и прямым взаимодействием с ядром, однако в будущем хотелось бы рассмотреть более сложные сценарии синхронизации процессов и работу с разделяемой памятью.